ImPRESS: Iterative Prompt Refinement through Semantic Slider Feedback and Foundation Model Guidance

Ken Maradiaga Rosales, Mathijs v. Sprang, and Koen Veldhorst

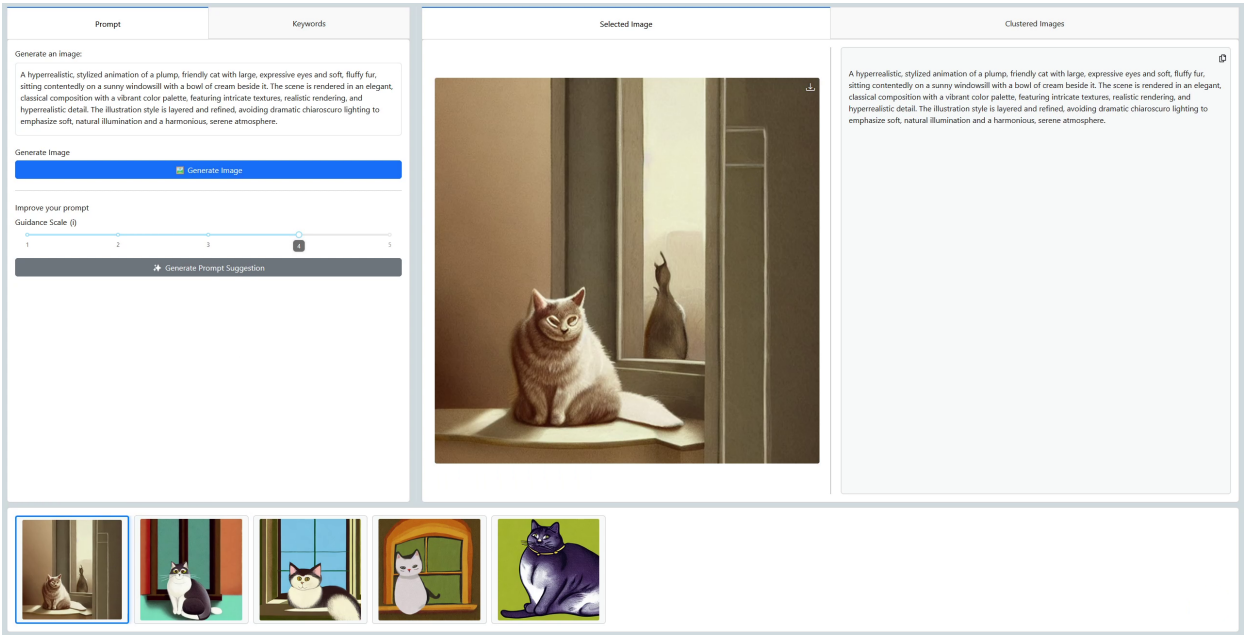

Fig. 1: A screenshot of the ImPRESS dashboard showing the prompting (left) and selected image (right) tabs, as well as the history store (bottom). Not shown are the keyword (left) and clustered view tabs (right). It can be seen that the user has gone through multiple iterations of image generation, which can be done by manually writing new prompts or by generating them (based on keywords).

Abstract—
Text-to-image generation models such as Stable Diffusion v1.5 have achieved remarkable image quality, but users still face a tedious, linear trial and error process without access to any prompt history or semantic cues. We introduce ImPRESS (Iterative Prompt REfinement through Semantic Slider feedback), a web-based dashboard that clusters 10 000 images DiffusionDB images using a combination of CLIP and UMAP, with interactive keyword sliders and LLM-based prompt suggestion using Qwen3-4B. ImPRESS tracks generated prompt-image pairs in a persistent history store to allow for backtracking and non-linear exploration. Furthermore, the dashboard visualizes the generated outputs and references them in a reactive 2D projection, showing where your outputs sit in semantic space. Temporal evaluation on an NVIDIA A100 shows that each refinement iterations takes about 13 s on average. Early qualitative analyses demonstrate that tweaking semantic sliders effectively shifts images towards user-selected clusters. ImPRESS is modular in its generative models, publicly available, and sets the stage for future user studies and potential model enhancements.

1 Introduction

In recent years, text-to-image generation models have steadily increased in popularity, being seen in both the academic world [23], as well as through widespread public use [16]. Such generative models take in textual prompts written by users as input, and use them to generate high-resolution images. State-of-the-art models are trained on large datasets of images and can thus create lifelike images in a wide variety of settings [37]. Further increases in the quality of the generated images can be accomplished by improving the training datasets [15], but this requires much laborious work and is compute- and memory intensive. Another way to increase the generated image quality is by improving the text-to-image generation model itself, which is the reason for a constant stream of research papers trying to make improvements to existing models [44]. Improvements in the generative models are very effective at improving the generated images themselves, but only focusing on the generative model ignores another way to improve output quality, which is how users can effectively communicate their intents to the model [22]. Prior works have successfully created visual analysis systems (VAS), such as PromptMagician [8], to aid users in the writing of prompts by visualizing their in- and output in a clustered semantic space among a larger dataset. Thus these VAS aim to visually show the relevant information for prompt-writing to guide users in their selection-process. Still, even with the aid of these systems, writing prompts requires many rounds of slowly fine-tuning the prompt to reach a good outcome. This iterative process can become messy and confusing to users when a history is not kept, as it becomes hard to remember which prompts were used to generate specific images.

- *Ken Maradiaga Rosales is with University of Amsterdam. E-mail: ken.maradiaga.rosales@student.uva.nl*
- *Mathijs v. Sprang is with University of Amsterdam. E-mail: mathijs.van.sprang@student.uva.nl.*
- *Koen Veldhorst is with University of Amsterdam. E-mail: koen.veldhorst@student.uva.nl.*

Currently the VAS do not keep a history, making the trial-and-error process very linear and limiting the options that users get, as they can not easily return to a prior result. Another area where current VAS come up short is that currently a user gets much information, giving them many areas to explore for prompt improvement, but they are still required to manually interpret this information and convert it to a new prompt. This manual reflection and repeated prompt writing is both arduous and error-prone. In this paper we aim to make the prompt-selection process for text-to-image generative models easier and faster by introducing a new VAS in ImPRESS (Iterative Prompt REfinement through Semantic Slider feedback), which keeps a clear history and automatically suggests prompt improvements using a Large Language Models (LLM). The layout and design of ImPRESS are shown in Figure 1. Thus this paper makes three main contributions:

1. Introduction of ImPRESS, a dashboard which allows for iterative prompt refinement for image generation.
2. History store implementation, which makes the iterative prompt writing more clear by keeping prior iterations in reach.
3. Automatic prompt suggestion, which uses an underlying LLM to incorporate user-specified keywords into the previous prompt.

ImPRESS allows users to view their generated image-prompt pairs in a clustered semantic space together with 10K images obtained from the DiffusionDB dataset [38]. This dataset contains a wide variety of generated images along with their prompts. ImPRESS also incorporates an underlying LLM to give prompt suggestions and extract keywords from the chosen cluster. In this paper we make use of the Qwen3-4B model [27]. ImPRESS also allows for native image generation within the dashboard, for this purpose the model used in this paper is Stable Diffusion v1.5 [1]. ImPRESS has been built with modularity in mind, and allows for an easy swap between underlying models used, as well as easily allowing for dataset alterations. In this manner the source code of ImPRESS has been made publicly available through GitHub.

2 Related Works

This work intersects several areas within the field of Artificial Intelligence, including natural language processing, image generation, prompt/image annotation, keyword extraction, and clustering. A key component of this project is its foundation in Multimedia Analytics, particularly building upon the framework proposed by Worring et al. [40]. Their model outlines a foundation-model-based paradigm for interactive analysis and feedback, which serves as the conceptual basis for ImPRESS. This framework enables a unified interface for prompt-to-image generation, semantic exploration, and iterative refinement, all driven by large-scale pre-trained models.

2.1 Prompt Keyword Extraction

ImPRESS has been developed to tackle the laborious process of finding a suitable prompt to generate a specific image. To help in this endeavor it extracts keywords from a large dataset of image-prompt pairs. This keyword extraction process is essential to quickly understanding the core concepts of large bodies of texts [9], and enables evaluation on the relation between similar or different image-prompt pairs [25]. Keyword/phrase extraction is inherently a problem regarding Information Retrieval and Natural Language interpretation [24], and is a classic problem that has led to various methods that approach keyword extraction unique angles [2]. Examples of keyword extraction approaches are: Support Vector Machine-based keyword extraction [43], graph-based keyword extraction [3], linguistics-based keyword extraction such as lexical chains [6], LLM-based keyword extraction [18] and many others. ImPRESS makes use of KeyBERT [11] for the keyword extraction process [11]. This is an easy-to-use and lightweight technique leveraging the cosine-similarity of BERT-embeddings to extract the key phrases and words from bodies of text.

2.2 Text-to-Image (TtI) Generation

Generative Artificial Intelligence (GAI) is currently a prominent topic within AI research, both as a subject of study [36] and as a tool applied in various research domains [10]. Many of these GAI models make use of prompts to generate lifelike images as a generated visual output [4]. One category of GAI models that use prompts to generate image are the diffusion models [42]. Models such as DALL-E [5, 29], Midjourney [20] or Stable Diffusion [17, 30] are examples of widely used and publicly-available diffusion models. For such diffusion models the prompt acts as an essential tool, letting the user convey their vision to the diffusion model. The quality and nature of the prompt is therefore essential to the final product [26]. Research has shown that by iteratively refining prompts that users can achieve more closely resembling their intentions [35].

2.3 Visual Analysis Systems for TtI

As outlined in the opening paragraph, the field of Multimedia Analytics is central to the design of ImPRESS and serves as the foundational framework upon which the system is built. In essence ImPRESS can be called a dashboard [31], allowing users to make data-driven choices regarding concerning the TtI process. Thus ImPRESS is an example of a VAS that tries to bridge the semantic gap between user and system [32]. It is not the first VAS that tries to make the prompting process more streamlined and accessible to users. Systems such as PromptMagician [8] or PromptIDE [33] have existed for some time now, and try to make the iterative prompt refinement process easier utilizing various techniques. PromptMagician in particular helps the prompt-writing process by guiding users with data-driven suggestions, dataset examples, and visual and textual information. It shows the images in a clustered, in dimensionality-reduced, semantic space. Whilst such existing systems are refined, they are not completely all-encompassing. Two areas where existing VAS for prompting lack are the absence of generated image / used prompt history, and the need for manual interpretation of information and its assimilation into a new prompt. Research shows that a VAS can greatly benefit form automatic prompt generation through LLMs, PromptAid is an example implementation, but it does not incorporate user-specified data-driven prompt recommendations [21, 39].

2.4 Importance of Prompt History

Prompting for image generation is a trial-and-error process, often requiring many iterations of small changes to come to a satisfactory result. By often changing the prompts used to generate images, it may become unclear which changes were applied and when, and how these changes affected the resulting image generation. It is thus very beneficial to users to maintain a clear overview of which prompts they have tried previously, and their outputs. In general, research has shown there are many different ways to incorporate histories into information visualization systems [13]. By incorporating and visualizing history in an intuitive manner, users of a system can more easily place their actions in context, which results in systems that are easier to use. There has been prior work on the effect of a history in GAI, such as PrompTHis [12]. PrompTHis shows the prompting history, as a graph-based network which uses clouds of images as nodes, and the differences between their prompts as edges. Through user studies PrompTHis shows the beneficial impact of a clear overview on the history of the prompting process.

3 Methodology

We present a unified multimedia analytics system, ImPRESS, that combines and extends existing techniques in navigation, guidance, and dimensionality reduction (DR) to support iterative prompt refinement and LLM-assisted guidance. This system enables users to interactively navigate generated images, modify text prompts through intuitive controls (such as semantic keyword sliders), and explore the refinement process through visual feedback and automated suggestions. The overall methodology involves application development, integration of generative and analytical models, and the implementation of interactive visual analytics techniques. The end-to-end flow of data and user interaction is summarized in Fig. 3.

3.1 Application Development

ImPRESS is developed as an interactive web-based dashboard built using the Dash and Plotly framework in Python. This framework was chosen for its flexibility in creating analytical applications with rich interactivity and high-quality visualizations. One advantage is that it enables development entirely in Python, which allows us to seamlessly integrate foundation models directly into the pipeline, without the overhead of learning or maintaining additional web development languages. The application takes inspiration from, modifies, and extends a base framework provided by the Multimedia Analytics (MMA) course, available at this GitHub repository.

The user interface consists of multiple components: text input fields for user prompts, image display panels for generated content, and interactive elements (e.g., popups and sliders) for exploring and refining the output space.

3.2 DiffusionDB Dataset

As part of our pipeline we use the DiffusionDB dataset [38], the first large-scale prompt-to-image dataset generated using Stable Diffusion. It contains 14 million images, each accompanied by their corresponding text prompt and generation parameters, all specified by real users. DiffusionDB provides two primary subsets: **2M**, which contains two million images in PNG format, and **Large**, which includes the full 14 million images stored in a lossless WebP format.

DiffusionDB also provides 16 predefined configurations derived from their primary subsets, varying by sample size (first-N or random-N instances) and format (standard or large). For our application, we selected the first 10,000 images from the DiffusionDB Large subset. This choice was motivated by the higher compression efficiency and lossless quality of the WebP format compared to PNGs, allowing for reduced storage size without compromising on quality. The selection totals approximately 4.5GB, offering a wide range of user-generated prompt-image pairs, while remaining small enough for a responsive experience and manageable preprocessing.

3.3 Integration of Diffusion Models

For prompt-to-image generation we integrate the Stable Diffusion v1.5 model [1] into our system, enabling the transformation of user-provided text prompts into synthetic images. This model was selected for its relatively lightweight architecture and fast inference capabilities. Upon receiving a prompt, the model generates corresponding visual content, allowing users to inspect how changes in the text prompt affect the output. This tight feedback loop between prompt input and image output supports an iterative and exploratory workflow, which is central to the design of our system.

The v1.5 model does have some limitations, and newer, more advanced open-source diffusion models have become available [7, 14]. However, these models typically demand significantly more computational resources, which given the experimental and demonstrative nature of this project, deemed them impractical. Nevertheless, the modularity of our codebase allows users to substitute the current Stable Diffusion model with higher performing alternatives.

3.4 Prompt Refinement Using LLMs

To enhance user creativity and improve interaction quality, we integrated the Qwen3 Large Language Model (LLM) [34] into the system to assist with prompt refinement. Specifically, we utilize the 4-billion-parameter variant [27], which offers a strong balance between computational efficiency and language generation capabilities.

Based on a user's initial input, the LLM provides prompt suggestions that include semantic variations, rewordings, or stylistic improvements, offering guidance tailored to the user's intended visual outcome. This is achieved through a combination of a user-defined guidance level and carefully constructed prompt templates, aiming for both flexibility and consistent output. The integration supports a human-in-the-loop workflow, ensuring that users retain control over the creative process while benefiting from the generative and analytical power of the LLM.

Prompt suggestions facilitated by the LLM can be automatically applied, allowing users to compare outputs side-by-side and iteratively refine their queries with the guidance of available models. This feature is central to enabling guided exploration and enhancing the usability of diffusion-based generation systems.

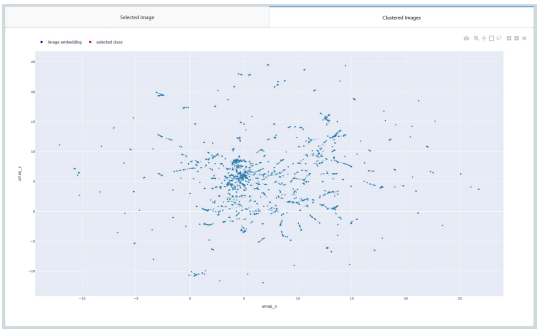

Fig. 2: UMAP projection of 10 000 reference images (blue) and the currently selected generated image (red). Semantically similar content form visual clusters.

3.5 Image Clustering and Similarity Analysis

To facilitate better navigation and understanding of the generated content, we implemented image clustering and similarity-based exploration techniques. We employ CLIP (Contrastive Language-Image Pretraining) [28] to embed generated and dataset images, alongside their respective prompts, into a shared latent feature space. These embeddings allow us to measure semantic and visual similarity between images.

To visualize the structure of this high-dimensional space, we applied UMAP (Uniform Manifold Approximation and Projection) [19], a non-linear manifold learning technique for dimension reduction. UMAP projects the image embeddings into a 2D space where similar images are grouped closer together, forming visually interpretable clusters. This is illustrated in Fig. 2. These clusters help users identify patterns, explore variations within and across prompt themes, and discover novel image regions that align with their creative intent.

3.6 Extracting Prompt Guiding Keywords

To extend the prompt refinement process beyond the user's initial input, we introduce an alternative strategy driven by visual cluster selection, rather than free-form text input. The UMAP projected clusters not only aid in navigation, but they also serve as a tool for context-aware prompt guidance. To achieve this, users may select a cluster of semantically similar images, or even make broader selections of multiple cluster.

Subsequently, the prompts associated with the selected images are aggregated and processed using KeyBERT, which identifies and extracts key terms based on BERT-embedding similarity. This results in a list of dominant concepts that characterizes the selected area. However, as these terms are derived from textual data alone, they may not always reflect concepts that are semantically meaningful for image generation. To address this, the extracted keywords are first processed by the Qwen3-4B language model, which filters and refines the list, aiming for keywords with greater relevance and coherence. The resulting set of keywords then form a basis for further prompt refinement.

For details on the prompt templates and formatting strategies used for LLM-guided prompt refinement, LLM-based keyword processing, and keyword-guided prompt modifications, please refer to Appendix A.

4 Interaction Design

The interaction design of ImPRESS is centered around facilitating exploration of image generation with prompt refinement so that users can get a visual understanding of image generation and its corresponding prompt. Every component has been designed to support multiple screen sizes. The system enables users to interact with generated image-prompt pairs in a semantic space populated with over 10,000 images from the DiffusionDB dataset [38]. This dataset offers a broad representation of prompts and corresponding synthetic images, enabling meaningful comparisons and contextual understanding.

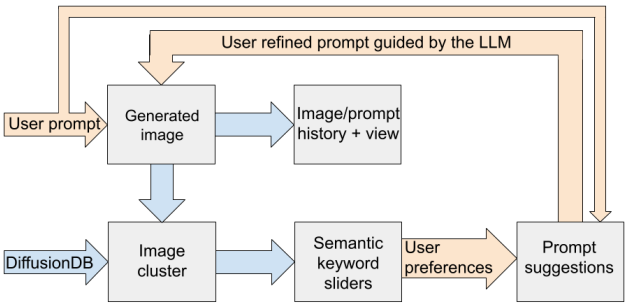

Fig. 3: This image shows the interaction pipeline interwoven into the main features of ImPRESS. The gray boxes show data that is visualized for the user, the orange arrows show the main user interactions and the blue arrows show the flow of ImPRESS. The figure shows that the user mainly interacts with ImPRESS by inputting a starting prompt, which ImPRESS uses to show similar prompt-image pairs, after which the user chooses the keywords to incorporate into the prompt, which ImPRESS generates automatically via an LLM from cluster selections.

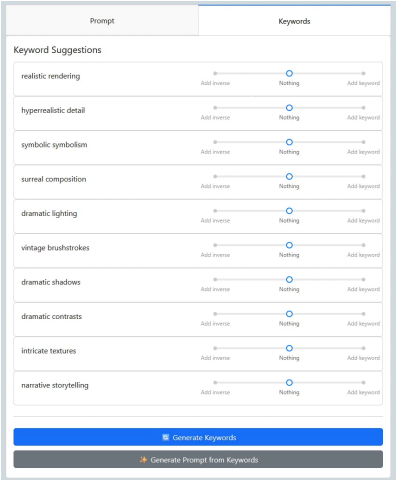

Fig. 4: The keyword-slider panel: users can promote, ignore, or invert each extracted term via a three-position control.

4.1 Image Generation

The initial interface presents users with the option to begin by generating an image from a custom prompt. Once generated, the image is displayed in a large main view alongside the corresponding prompt, while a history panel beneath tracks the evolution of generated images over time as the prompt is modified and new images are generated. This allows users to visually compare different stages of their creative process and to iteratively explore how their images evolve with different prompts. During image generation the button will be blocked and both on the main view as in the button a loading spinner will appear blocking any more requests from the user and to let the user know that an image is being generated.

4.2 LLM Prompt Refinement

At this stage, users can begin refining their prompt using a "guidance scale" control, which determines how significantly the original prompt should be altered. This refinement request is sent to an integrated LLM. The LLM generates a revised version of the prompt, which is presented in a pop-up modal. Within this modal, users can either accept the LLM's suggestion, automatically updating the prompt, or reject it, preserving their original input. The modal will show up immediately and while the user waits a waiting message is displayed to let the user know the LLM is generating a new suggestion.

4.3 Image Clustering

ImPRESS leverages dimensionality reduction and semantic clustering to present both generated and reference images in a coherent two-dimensional layout. Within the dedicated "Clusters" view, users can see their generated image positioned alongside 10,000 reference images from the DiffusionDB dataset, as well as other previously generated outputs. This spatial representation is derived from shared visual-semantic embeddings, allowing similar images to appear in close proximity making it easy for the user to find similar images. Images are only shown when the user has zoomed in enough and only 100 images or less are in the view optimizing the resources of the client and to prevent the view from being too cluttered.

4.4 Keyword Prompt Refinement

Users can interactively explore the 2D semantic space by zooming into regions of interest and selecting image clusters. To aid orientation, the user's generated image is distinctly highlighted in red, allowing for quick identification and contextual comparison with semantically similar images from the dataset. This visualization supports users in understanding the relationships between their prompt and nearby image content. Upon selecting a relevant cluster, users can click the "Generate Keywords" button to retrieve a set of ten keywords derived from the selected images. These keywords serve as semantic descriptors and can be interactively configured using a slider-based interface. For each keyword, the user can choose to include it, ignore it, or treat it as a negative prompt modifier. An illustration of this panel can be view in Fig. 4. Once the keyword configuration is complete, users can send the selected inputs to the integrated Large Language Model (LLM), which generates a revised prompt based on the refined semantic criteria. As with previous LLM interactions, the new prompt is presented in a pop-up modal, where users can choose to accept or reject the suggestion before proceeding with image generation.

5 Implementation

As illustrated in the interaction pipeline (Fig. 3), users may trigger a series of model inferences and visual updates for the interface. This section describes the key architectural components and callback mechanisms that power the interface.

5.1 Architecture Overview

The system is organized into a modular dashboard composed of three main user-facing components:

- **Input Panel** — Enables users to enter prompts or select from automatically extracted keywords.
- **Image View Panel** — Displays either the currently generated image or a clustered projection of all generated outputs.
- **History Panel** — Maintains a scrollable record of all previously generated images, allowing users to revisit and reuse past prompts.

These panels are supported by a series of backend processes responsible for computational tasks, including:

- **Image Generation** using the Stable Diffusion pipeline.
- **Prompt Refinement** via a language model (LLM) based on user guidance or keyword selection.
- **Keyword Extraction** powered by the KeyBERT algorithm, which analyzes prompt content to surface stylistic or thematic terms.
- **Projection and Visualization** using dimensionality reduction techniques such as t-SNE and UMAP to plot image embeddings in an interactive scatterplot.

The entire application is orchestrated using **Dash callbacks**, which form the backbone of interactivity. These callbacks are event-driven functions that react to user actions, clicking a button or selecting an image, and dynamically update the corresponding state, visuals, or interface components in real time.

5.2 Stable Diffusion Integration

Image generation in the dashboard is powered by stable-diffusion-v1-5 [1]. This functionality is initiated via a button, which triggers a Dash callback responsible for orchestrating the full generation pipeline. The implementation is carefully designed to ensure both concurrency safety and real-time UI responsiveness.

1. **Thread-Safe Access to the Pipeline:** To prevent simultaneous access to the GPU by multiple threads, a global lock (`pipe_lock`) is acquired before any image generation begins. This ensures safe and exclusive access to the pipeline object on either CUDA or MPS backends.
2. **Prompt-Conditioned Generation:** Once locked, the pipeline processes the user-provided prompt using the stable-diffusion-v1-5 model. Image generation is parameterized by a fixed number of inference steps (50) and a single image output per prompt.
3. **Post-Processing and Encoding:** The generated image is resized to a standardized resolution, converted to PNG format, and encoded into a base64 string for efficient transmission and rendering in the browser. This is handled in-memory using Python's `BytesIO` and PIL libraries, avoiding any disk I/O overhead.
4. **Embedding and Projection:** The image, along with its associated prompt, is embedded using a CLIP-based model. The resulting joint embeddings are projected into a 2D latent space using a pre-trained UMAP model [19]. This projection is appended to the scatterplot, visually situating the new image within a semantic neighborhood of previously generated content.
5. **State Management and History Tracking:** A unique identifier is assigned to each image using UUIDs. Metadata like the prompt, projection coordinates, and base64 image is stored in the history-store, a persistent client-side container. This enables immediate re-selection and download, and supports interactive clustering in subsequent visualizations keeping the backend stateless.

5.3 Scatterplot and Selection

To provide a semantic overview of all generated images, each image–prompt pair is embedded into a shared latent space using a CLIP-based model. These embeddings are then projected into two dimensions using a pre-trained UMAP model, allowing for visual clustering based on stylistic or thematic similarity. The 2D projection is rendered as an interactive scatterplot using Plotly, with newly generated images appended dynamically to the figure's data. Thumbnails of the images are embedded directly on the plot. To maintain performance and avoid visual clutter, this function selectively overlays base64-encoded image previews based on the current zoom window. A maximum cap, of 100 images, ensures that rendering remains efficient even as the number of images increases.

5.4 Keyword and Prompt Engineering

The system implements two complementary mechanisms for prompt enhancement, both relying on transformer-based language models: one using direct instruction-based rewriting and the other leveraging extracted style keywords.

The first mechanism utilizes the language model Qwen3-4B [27]. The model is loaded lazily using a singleton pattern to avoid repeated initialization and is deployed with automatic device mapping and evaluation mode enabled for efficiency. Prompt enhancement is guided by a numeric scale (1–5) that determines the rewrite strength. This guidance level is mapped to a predefined textual instruction, which is concatenated with the original prompt and passed to the model using a chat-based input template A.1. The LLM response is extracted using token-based heuristics and returned as the improved prompt.

The second mechanism extends this capability by grounding the improvement in visual-semantic context. When a user selects multiple images from the scatterplot, the associated prompts are aggregated and passed to the `KeyBERT` model to extract salient keywords. These keywords are then filtered for stylistic relevance, a meta-instruction is formulated, and the same underlying LLM is reused. A structured prompt is constructed that specifies how to include or invert particular keywords. The model is then queried to return a prompt that semantically incorporates the selected visual features.

Both modes rely on the same inference backend and tokenizer logic, and their modular implementation allows for scalable extension or substitution of models. Outputs from both processes are returned as plain text and can be directly substituted into the prompt state or stored for future reuse. This decoupled architecture ensures prompt engineering capabilities remain flexible and model-agnostic.

5.5 History and State Synchronization

The application maintains a comprehensive interaction history using Dash's client-side storage mechanism, specifically the `dcc.Store` component. All generated outputs including the image (in base64 format), the corresponding prompt, and the projection coordinates are appended to a list stored in the `history-store`. Each entry is uniquely identified using a UUID and preserved in memory for the duration of the session.

The visual representation of the history is managed by a callback that updates the `history-wrapper` container. For each historical entry, a thumbnail preview is rendered using a `dbc.Card` component, with an associated `dbc.Tooltip` displaying the prompt text. When a user selects a past image by clicking a thumbnail, the corresponding entry is retrieved from the store, parsed, and used to:

- Update the Image View Panel by reconstructing the image using its base64 representation.
- Re-synchronize the scatterplot by identifying and highlighting the associated projection coordinates.
- Repopulate the prompt text for further editing or reuse.

A separate callback enables image downloads by decoding the selected image's base64 string into binary format and returning it via the `dcc.Download` component. This functionality is stateless and does not persist files server-side, ensuring secure, ephemeral delivery.

5.6 Dynamic UI Controls

User feedback and interface responsiveness are managed through Dash's reactive components. Button states are controlled via `dcc.Store` flags that disable inputs during processing and display Bootstrap-based loading spinners. `dcc.Loading` wrappers provide visual feedback for longer operations such as image generation or keyword extraction. Prompt tools are shown conditionally based on input validity, and `dcc.Clipboard` enables one-click copying of prompts. These behaviors are coordinated through stateless callbacks, ensuring consistent and responsive UI updates.

6 Evaluation

Due the multi-modal nature of multimedia analytics systems, such as ImPRESS, the can be evaluated among many different axis. This paper evaluates the newly introduced VAS ImPRESS by measuring inference speed among various pipeline components. ImPRESS is also evaluated in a qualitative manner by looking at the reduced CLIP embeddings of images generated from the keyword-based recommendations compared to the original prompt generations. Due to time-constraints this paper was not able to perform user-studies on the workability of ImPRESS.

6.1 Temporal Evaluation

To measure the temporal quality of ImPRESS, this paper isolates every component in the ImPRESS pipeline. By measuring the average inference time of these components, they can be added up to simulate a single iteration of prompt refinement. By comparing it to user statistics on the average and median amounts of prompt refinement iterations, an estimation can be made on the time it takes to reach a satisfactory outcome. The reported times were run on a singular NVIDIA A100-SXM4-40GB part of the Snellius cluster. As the results were not obtained locally, the reported "miscellaneous" time in Table 1 is inflated due to latency.

Pipeline component	Runtime (ms)
Image generation	3894±439
Image clustering	4239±1387
Keyword extraction	2984±897
Prompt generation	1320±114
Miscellaneous	948±89
Total iteration	13385±1698

Table 1: Average time and standard deviation for specific parts of the ImPRESS pipeline to complete. The numbers are shown in milliseconds and are averaged out over 5 runs. The time it takes for users to interact with the dashboard are not included, as no careful user study has been performed. These numbers may be inflated as ImPRESS was run on the snellius cluster, adding extra latency.

In Table 1 the average execution times and the standard deviation over 5 separate runs are shown. It can be seen that one single iteration of prompt improvements takes around 13 seconds, on top of the time it takes for users to interpret data and interact with the system. Using data on user statistics for DiffusionDB, taken from Xie et al. [41], the average iterations per session of 13.71 would take roughly three minutes of execution time, added on top of user interaction time. The bulk of this time is taken up by the various models used, and will thus likely vary significantly for different models.

6.2 Qualitative Evaluation

To measure a qualitative aspect of ImPRESS, we look at how well the prompts generated from the selected keywords manage to shift the generated image in semantic space. The embeddings are generated via CLIP and the distance is calculated using the dimensionally-reduced euclidean distance of said CLIP embeddings. The distance of the generated image is measured compared to the center of the selected cluster, used to generate the embeddings for subsequent prompt alterations. One run starts with a unique single-sentence prompt, to generate a new image. A singular cluster is then selected based on similarity in size and distance to the generated image. Then by extracting keywords and positively adding 5 random keywords to the prompt generator, a new image is generated. By returning to the original prompt, images for 10 positive keyword additions, 5 inverse additions and 10 inverse additions are also ran. Then a new run is started by writing a new original single-sentence prompt, and selecting a new cluster.

Prompt type	Semantic space distance
Original prompt	16.386±1.294
5 Positive keywords	4.891±4.047
10 Positive keywords	4.873±3.984
5 Inverse keywords	14.896±3.404
10 Inverse keywords	14.698±3.571

Table 2: The average distance and standard deviation from the generated image to the selected cluster center. The clusters were selected manually, which were chosen based on a similar distance to the first generated image. The distance shown is calculated by taking the euclidean distance in 2-dimensional CLIP embeddings semantic space. The numbers are the resultant of 5 separate runs.

In Table 2 it can be seen that the addition of 5 or 10 keywords does not matter too much, as both effectively close the distance in semantic space to the chosen cluster. The reported numbers are the resultant of 5 separate runs, as they were described above. The average distance shows that both the positive and the inverse keyword additions move closer to the chosen cluster. By looking at the standard deviation, it can be seen that the results varied greatly. In the individual keyword runs it could be seen that some newly generated prompts moved very close to the clusters, as close as 1.672 units away, whilst others did not move nearly as much, but all moved closer to the chosen cluster. For the inverse keyword additions some stayed near their starting point, but in one run both the 5- and 10 keyword versions moved closer to the cluster by a fair margin, thus skewing the averages. After inspection of the newly generated prompts, some inverse keywords were added in a positive manner. This suggests that the underlying LLM does not always correctly interpret the instruction prompt, which may be solvable by using a stronger LLM or tuning the instruction prompt. In general, there was no significant difference between the 5- and 10-keywords addition runs for both the inverse and positive cases.

7 Conclusion

ImPRESS is a visual analysis system which allows users to generate images by inputting prompts and given data-driven suggestions. Through qualitative evaluation it is possible to see that the positive keyword additions significantly alter the CLIP embeddings, moving them closer to the cluster from which the keywords were extracted. The effect of the keywords on the position in semantic cluster space, however, vary greatly. This can attributed to the stochastic nature of both text-to-image generation models and LLMs, this high variation can also be attributed to the great dimensionality reduction that is performed in semantic space or to the fact that prompts can be more closely related to some clusters than others. The addition of inverse keywords also shows a slight movement into the direction of the selected cluster in semantic space. This is not an expected result, but it can be explained by the fact that the LLM does not always correctly add inverse keywords and by the fact that the prompt generation based on keywords is highly stochastic. More often than not the inverse keywords did not greatly effect the position of the generated image in semantic space. There also did not seem to be a significant correlation between the number of specified keywords used for prompt generation, and the position of the resultant image.

By evaluating the average execution times of various parts of the ImPRESS pipeline it can be shown that most of this time is used by the used underlying models. The miscellaneous time used by the ImPRESS dashboard in Table 1 is comparatively small, and may be inflated due to remotely evaluating the system. Following this, it can be assumed that using ImPRESS does not take significantly more execution time than the standalone models. By automatically generating prompt suggestions the total time it takes to generate a final image will likely be smaller, but this can't be concluded with current results.

In general it can be concluded that ImPRESS successfully allows users to generate images and helps them in their iterative prompt refinement process. The history store works in theory, but as no user study has been performed due to time constraints, its true effectiveness remains unclear. The automatic prompt refinement through (keyword-based) LLM suggestions can be concluded to work, as the position of the generated image in semantic embedding space moves in the intended direction.

8 Discussion

The implementation of ImPRESS has been tested both on an NVIDIA A100-SXM4-40GB GPU on the Snellius cluster, as well as a RTX 4080 GPU. These set-ups greatly limited the choice in models, only allowing for for small models due to memory-constraints. Thus, by using a smaller and weaker LLM and Diffusion model, the current state of ImPRESS discussed in this paper is more akin to a floor than a ceiling on the effectiveness of the dashboard implementation.

Due to time-constraints, the evaluation of ImPRESS has not been thorough. With only 5 runs for both the temporal and qualitative evaluation. Due to the highly stochastic nature of the underlying models, the results show a high variance. Thus by performing more runs the conclusions would be more substantiated. Also, a user-study could not be performed as the implementation was not finished in time to gather and test on various users. This greatly limits the number of conclusions that can be drawn about the dashboard, for example leaving the effect of the history store up to interpretation.

8.1 Future Work

As mentioned above an interesting starting point for future work would be to incorporate a user study on the current state of the ImPRESS system. This would allow for a more nuanced look on the design and

implementation of the dashboard, and whether it achieves the goals of leveraging the addition of a clear history and usable prompt suggestions to make the prompt refinement easier and faster.

Another straightforward improvement that can be made in future work is to add newer and stronger underlying models in the LLM and image generation models. This would allow for higher-quality results and a look into whether the automatically generated keyword-based prompts also work effectively for stronger models. This would also test the modularity of the ImPRESS.

When compared to existing VAS ImPRESS comes short in some areas, but also introduces a few new concepts. The shortcomings can largely be attributed to the nature in which the research was performed, as part of the multimedia analytics course, which required some interesting fields and additions to be passed over. For such reasons the history store was kept basic, without clear relations between the used prompts and generated images. An interesting topic for future works would be to incorporate more complex history stores, such as the graph based method of PrompTHis [12]. Also, some user-inputs are currently not implemented, for example the addition of a negative prompt for the image generation. The underlying models allow for the addition of these additional parameters, but the current state of ImPRESS ignores them. Some existing VAS, for example PromptMagician [8], already implement some of these features. An interesting point of future work would be to add more user parameters, present in both the image generation and clustered image space.

References

[1] S. AI. Stable diffusion 1.5. https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5. Accessed: 2025-06-26. 2, 3, 5

[2] S. Beliga. Keyword extraction: a review of methods and approaches. *University of Rijeka, Department of Informatics, Rijeka*, 1(9):1–9, 2014. 2

[3] S. Beliga, A. Meštrović, and S. Martinčić-Ipšić. An overview of graph-based keyword extraction methods and approaches. *Journal of information and organizational sciences*, 39(1):1–20, 2015. 2

[4] S. Bengesi, H. El-Sayed, M. K. Sarker, Y. Houkpati, J. Irungu, and T. Oladunni. Advancements in generative ai: A comprehensive review of gans, gpt, autoencoders, diffusion model, and transformers. *IEEE Access*, 12:69812–69837, 2024. doi: 10.1109/ACCESS.2024.3397775 2

[5] J. Betker, A. Ramesh, and et al. Improving image generation with better captions. OpenAI technical report, 2023. Available at https://cdn.openai.com/papers/dall-e-3.pdf. 2

[6] G. Ercan and I. Cicekli. Using lexical chains for keyword extraction. *Information Processing & Management*, 43(6):1705–1714, 2007. 2

[7] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, K. Lacey, A. Goodwin, Y. Marek, and R. Rombach. Scaling rectified flow transformers for high-resolution image synthesis, 2024. doi: 10.48550/arXiv.2403.03206 3

[8] Y. Feng, X. Wang, K. K. Wong, S. Wang, Y. Lu, M. Zhu, B. Wang, and W. Chen. PromptMagician: Interactive prompt engineering for text-to-image creation. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):295–305, Jan. 2024. doi: 10.1109/tvcg.2023.3327168 1, 2, 7

[9] N. Firoozeh, A. Nazarenko, F. Alizon, and B. Daille. Keyword extraction: Issues and methods. *Natural Language Engineering*, 26(3):259–291, 2020. doi: 10.1017/S1351324919000457 2

[10] C. Ganjavi, M. B. Eppler, A. Pekcan, B. Biedermann, A. Abreu, G. S. Collins, I. S. Gill, and G. E. Cacciamani. Publishers' and journals' instructions to authors on use of generative artificial intelligence in academic and scientific publishing: bibliometric analysis. *bmj*, 384, 2024. 2

[11] M. Grootendorst. Keybert: Minimal keyword extraction with bert., 2020. doi: 10.5281/zenodo.4461265 2

[12] Y. Guo, H. Shao, C. Liu, K. Xu, and X. Yuan. Promptthis: Visualizing the process and influence of prompt editing during text-to-image creation. *IEEE Transactions on Visualization and Computer Graphics*, pp. 1–12, 2024. doi: 10.1109/TVCG.2024.3408255 2, 7

[13] J. Heer, J. Mackinlay, C. Stolte, and M. Agrawala. Graphical histories for visualization: Supporting analysis, communication, and evaluation. *IEEE Transactions on Visualization and Computer Graphics*, 14(6):1189–1196, 2008. doi: 10.1109/TVCG.2008.137 2

[14] B. F. Labs, S. Batifol, A. Blattmann, F. Boesel, S. Consul, C. Diagne, T. Dockhorn, J. English, Z. English, P. Esser, S. Kulal, K. Lacey, Y. Levi, C. Li, D. Lorenz, J. Müller, D. Podell, R. Rombach, H. Saini, A. Sauer, and L. Smith. Flux.1 kontext: Flow matching for in-context image generation and editing in latent space, 2025. 3

[15] H. Li, Y. Zou, Y. Wang, O. Majumder, Y. Xie, R. Manmatha, A. Swaminathan, Z. Tu, S. Ermon, and S. Soatto. On the scalability of diffusion-based text-to-image generation, 2024. 1

[16] V. Liu and L. B. Chilton. Design guidelines for prompt engineering text-to-image generative models. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, CHI '22, article no. 384, 23 pages. Association for Computing Machinery, New York, NY, USA, 2022. doi: 10.1145/3491102.3501825 1

[17] Lopez, Joshua and Stability AI. Stable diffusion 3: Multimodal diffusion transformer. Stability AI technical report / arXiv, 2025. Published March 5, 2025. 2

[18] R. Y. Maragheh, C. Fang, C. C. Irugu, P. Parikh, J. Cho, J. Xu, S. Sukumar, M. Patel, E. Korpeoglu, S. Kumar, and K. Achan. Llm-take: Theme-aware keyword extraction using large language models. In *2023 IEEE International Conference on Big Data (BigData)*, pp. 4318–4324, 2023. doi: 10.1109/BigData59044.2023.10386476 2

[19] L. McInnes, J. Healy, N. Saul, and L. Großberger. UMAP: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29):861, Sept. 2018. doi: 10.21105/joss.00861 3, 5

[20] Midjourney, Inc. Midjourney, version 1 (beta). Discord bot release, 2022. Open beta began July 12, 2022; latest model V7 released April 4, 2025. 2

[21] A. Mishra, U. Soni, A. Arunkumar, J. Huang, B. C. Kwon, and C. Bryan. Promptaid: Prompt exploration, perturbation, testing and iteration using visual analytics for large language models, 2025. 2

[22] W. Mo, T. Zhang, Y. Bai, B. Su, J.-R. Wen, and Q. Yang. Dynamic prompt optimizing for text-to-image generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 26627–26636, June 2024. 1

[23] A. Nichol, P. Dhariwal, A. Ramesh, P. Shyam, P. Mishkin, B. McGrew, I. Sutskever, and M. Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models, 2022. 1

[24] T. Nomoto. Keyword extraction: a modern perspective. *SN Computer Science*, 4(1):92, 2022. 2

[25] N. Pavlichenko and D. Ustalov. Best prompts for text-to-image models and how to find them. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '23, 5 pages, p. 2067–2071. Association for Computing Machinery, New York, NY, USA, 2023. doi: 10.1145/3539618.3592000 2

[26] J. Phoenix and M. Taylor. *Prompt engineering for generative AI*. " O'Reilly Media, Inc.", 2024. 2

[27] A. C. Qwen Team. Qwen3 4b. Accessed:2025-06-26. 2, 3, 5

[28] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision, 2021. doi: 10.48550/arXiv.2103.00020 3

[29] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever. Zero-shot text-to-image generation, 2021. 2

[30] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684–10695, June 2022. doi: 10.48550/arXiv.2112.10752 2

[31] A. Sarikaya, M. Correll, L. Bartram, M. Tory, and D. Fisher. What do we talk about when we talk about dashboards? *IEEE Transactions on Visualization and Computer Graphics*, 25(1):682–692, 2019. doi: 10.1109/TVCG.2018.2864903 2

[32] A. Smeulders, M. Worring, S. Santini, A. Gupta, and R. Jain. Content-based image retrieval at the end of the early years. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(12):1349–1380, 2000. doi: 10.1109/34.895972 2

[33] H. Strobelt, A. Webson, V. Sanh, B. Hoover, J. Beyer, H. Pfister, and A. M. Rush. Interactive and visual prompt engineering for ad-hoc task adaptation with large language models, 2022. 2

[34] Q. Team. Qwen3 technical report, 2025. doi: 10.48550/arXiv.2505.09388 3

[35] K. Trinh, S. Seidenberger, R. Wijewickrama, M. Jadliwala, and A. Maiti. A picture is worth a thousand prompts? efficacy of iterative human-driven prompt refinement in image regeneration tasks, 2025. 2

[36] C. G. Turhan and H. S. Bilge. Recent trends in deep generative models: a review. In *2018 3rd International Conference on Computer Science and Engineering (UBMK)*, pp. 574–579. IEEE, 2018. 2

[37] Z. Wang, Y. Jiang, H. Zheng, P. Wang, P. He, Z. A. Wang, W. Chen, and M. Zhou. Patch diffusion: Faster and more data-efficient training of diffusion models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, eds., *Advances in Neural Information Processing Systems*, vol. 36, pp. 72137–72154. Curran Associates, Inc., 2023. 1

[38] Z. J. Wang, E. Montoya, D. Munechika, H. Yang, B. Hoover, and D. H. Chau. DiffusionDB: A large-scale prompt gallery dataset for text-to-image generative models, 2022. doi: 10.48550/arXiv.2210.14896 2, 3

[39] Z. Wen, L. Weng, Y. Tang, R. Zhang, Y. Liu, B. Pan, M. Zhu, and W. Chen. Exploring multimodal prompt for visualization authoring with large language models, 2025. 2

[40] M. Worring, J. Zahálka, S. van den Elzen, M. T. Fischer, and D. A. Keim. A multimedia analytics model for the foundation model era, Apr. 2025. doi: 10.48550/arXiv.2504.06138 2

[41] Y. Xie, Z. Pan, J. Ma, L. Jie, and Q. Mei. A prompt log analysis of text-to-image generation systems. In *Proceedings of the ACM Web Conference 2023*, WWW '23, p. 3892–3902. ACM, Apr. 2023. doi: 10.1145/3543507.3587430 6

[42] C. Zhang, C. Zhang, M. Zhang, I. S. Kweon, and J. Kim. Text-to-image diffusion models in generative ai: A survey, 2024. 2

[43] K. Zhang, H. Xu, J. Tang, and J. Li. Keyword extraction using support vector machine. In *Advances in Web-Age Information Management: 7th International Conference, WAIM 2006, Hong Kong, China, June 17-19, 2006. Proceedings 7*, pp. 85–96. Springer, 2006. 2

[44] T. Zhang, Z. Wang, J. Huang, M. M. Tasnim, and W. Shi. A survey of diffusion based image generation models: Issues and their solutions, 2023. 1

A Prompt Templates

This appendix provides the full system and user prompt templates used in our keyword- and prompt-refinement workflows. Below we list the templates for guidance-scale rewriting, filtering KeyBERT extracted keywords, and incorporation of slider-based keywords.

A.1 Guidance Scale-based Prompt Refinement

The boxes below define the system and user prompt templates that are used with for LLM-based prompt refinement and rewriting at five discrete 'guidance' levels. The instruction corresponding to the selected level is parsed directly as the improvement instruction seen in the users prompt template.

System Prompt

Your role is to come up with a better prompt for the user to generate an image. You will respond with the new improved prompt only.

User Prompt Template

Based on the following prompt, come up with one that is better suited for generating an image: `{prompt_text}`.
Instruction: `{improvement_instruction}`
Respond only with the newly improved prompt.

Guidance Levels

- 1: Make a very small improvement to the prompt. Keep it mostly unchanged.
- 2: Make minor enhancements to improve clarity and creativity.
- 3: Refactor the prompt moderately for better structure and more impact.
- 4: Significantly improve the prompt while preserving its core idea.
- 5: Completely rewrite the prompt to maximize creativity and effectiveness.

A.2 KeyBert Keyword Filtering

The following illustrates part of the two-step prompt template used to filter and expand on extracted keywords. First, an example input-output pair is shown to set the format, after which the actual keywords are processed. The top_n parameter defaults to 10.

Example/User Prompt Template

I have the following list of keywords that were extracted from user prompts intended for image generation:
- `{', '.join(candidates)}`
Your task is to analyze these keywords and return a new list of words that relate to: *style*, *aesthetic*, *mood*, *genre*, or any other visually relevant concept implied or suggested by the input.
Focus on: Artistic styles (e.g., minimalism, surrealism, vaporwave), moods and atmosphere (e.g., serene, eerie, chaotic), visual characteristics (e.g., high contrast, muted colors, soft lighting), cultural or conceptual associations (e.g., cyberpunk, baroque, post-apocalyptic).
Avoid: Repeating the input keywords exactly, and listing generic or overly broad terms (e.g., "image", "art").
Only return the `{top_n}` most relevant keywords, separated by commas, without any additional text or explanation.

A.3 Keyword Sliders Processing

For the processing of information passed through the keyword sliders, we have defined the following templates. These templates specify how positive and negative keywords are formatted for LLM processing.

System Prompt

Your role is to come up with a better prompt for the user to generate an image. You will respond with the new improved prompt only.

User Prompt Template

Based on the following prompt, come up with one that is better suited for generating an image: `{prompt_text}`.
Instruction: `{improvement_instruction}`
Respond only with the newly improved prompt.

Slider Instructions

Incorporate the opposite of the following keywords:
`{', '.join(negatives)}`.
Change the prompt by incorporating the following keywords:
`{', '.join(positives)}`.
Stay close to the original prompt, don't change it too much.